SymPy Bug Report

1 Bug 20: Conditional Uniform Density Keeps the Wrong Support

1.1 Minimal Reproducer

```
from sympy import S, integrate, oo, symbols
from sympy.stats import P, Uniform, density, given

x = symbols("x")
U = Uniform("U", 0, 1)
Y = given(U, U > S.Half)
d = density(Y)(x)

print("density:", d)
print("total mass:", integrate(d, (x, -oo, oo)))
print("mass below 1/2 from density:", integrate(d, (x, 0, S.Half)))
print("P(Y < 1/2):", P(Y < S.Half))
```

1.2 Observed Behavior

```
density: 2*Piecewise((1, (x >= 0) & (x <= 1)), (0, True))
total mass: 2
mass below 1/2 from density: 1
P(Y < 1/2): 0
```

SymPy applies the factor $2 = 1/P(U > 1/2)$, but the returned density is still supported on the original interval $[0, 1]$. As a result, the expression returned by `density` is not a probability density: it integrates to 2 and assigns mass 1 to $[0, 1/2]$, even though the conditional probability query gives $P(Y < 1/2) = 0$.

1.3 Expected Behavior

For $U \sim \text{Uniform}(0, 1)$ and $Y = U \mid U > 1/2$, the conditional density should be

$$f_Y(x) = \begin{cases} 2, & 1/2 < x \leq 1, \\ 0, & \text{otherwise.} \end{cases}$$

Equivalently, the returned density should have total mass 1, zero mass below the conditioning threshold, and value 0 at any probe point $x < 1/2$.

1.4 Mathematical Explanation

The conditional density is obtained by restricting the original density to the conditioning event and then renormalizing:

$$f_{U|U>1/2}(x) = \frac{f_U(x)\,1_{\{x>1/2\}}}{P(U > 1/2)}.$$

Since $f_U(x) = 1$ on $[0, 1]$ and $P(U > 1/2) = 1/2$, this gives density 2 only on the restricted support $(1/2, 1]$.

SymPy's result instead represents $21_{[0,1]}(x)$. This is not an equivalent form of the conditional density: it is positive on points that violate the condition, and

$$\int_{-\infty}^{\infty} 21_{[0,1]}(x)\,dx = 2 \qquad \text{while} \qquad \int_{-\infty}^{\infty} f_{U|U>1/2}(x)\,dx = 1.$$

The discrepancy is therefore a support error and a normalization error, not a harmless simplification or alternate representation.

1.5 Source-Level Diagnosis

The diagnosis identifies the relevant implementation in `sympy/stats/crv.py`.

In that file, the relevant methods are `ContinuousPSpace.conditional_space`, around lines 446–462, and `ContinuousPSpace.compute_density`, around lines 333–340. The traced call path is

`sympy.stats.rv.given`
→ `ContinuousPSpace.conditional_space`
→ `Density.doit/density`
→ `ContinuousPSpace.compute_density`.

For the reproducer, the conditional probability space has a `ConditionalContinuousDomain` with set Interval. Lopen$(1/2, 1)$, so the domain restriction itself is computed. The stored density, however, remains the original uniform density divided only by the conditioning normalizer.

```
domain = ConditionalContinuousDomain(self.domain, condition)
norm = domain.compute_expectation(self.pdf, **kwargs)
pdf = self.pdf / norm.xreplace(replacement)
density = Lambda(tuple(domain.symbols), pdf)
return ContinuousPSpace(domain, density)
```

This rule renormalizes the original density but does not multiply it by an indicator for the conditional domain. The second part of the failure occurs when `density(Y)` requests the density of the same variable. In that case, `compute_density` has no random symbols to marginalize, and `ConditionalContinuousDomain.compute_expectation` returns the stored expression unchanged when the variable list is empty. The condition $U > 1/2$ is therefore never applied to the returned density, producing $21_{[0,1]}$ instead of $21_{(1/2,1]}$.

The diagnosis suggests that a plausible fix is to ensure that conditional continuous densities include the condition's support, either by incorporating an indicator or `Piecewise` restriction in `conditional_space`, or by making density extraction for a `ConditionalContinuousDomain` apply the domain restriction even when no variables are marginalized. A general fix would need to avoid double-applying the original distribution support already present in many distribution PDFs.

1.6 Additional Failing Instances

The same failure occurs for a family of conditional uniform densities $Y_t = U \mid U > t$ with $0 < t < 1$. For every such threshold, the correct density is $1/(1-t)$ only on $(t, 1]$ and 0 below t. The verification cases check both global normalization and local support by integrating the returned density over $[0, t]$ and evaluating it at $t/2$.

Input / Expression	SymPy behavior	Expected behavior
$Y = \text{given}(U, U > 1/2)$, $\text{density}(Y)(x)$	$2\mathbb{1}_{[0,1]}(x)$; total mass 2; mass on $[0, 1/2]$ is 1	$2\mathbb{1}_{(1/2,1]}(x)$; total mass 1; zero mass below $1/2$
$Y = \text{given}(U, U > 1/3)$, $\text{density}(Y)(x)$	$\frac{3}{2}\mathbb{1}_{[0,1]}(x)$; positive at $x = 1/6$	$\frac{3}{2}\mathbb{1}_{(1/3,1]}(x)$; zero at $x = 1/6$
$Y = \text{given}(U, U > 1/4)$, $\text{density}(Y)(x)$	$\frac{4}{3}\mathbb{1}_{[0,1]}(x)$; positive at $x = 1/8$	$\frac{4}{3}\mathbb{1}_{(1/4,1]}(x)$; zero at $x = 1/8$
$Y = \text{given}(U, U > 2/3)$, $\text{density}(Y)(x)$	$3\mathbb{1}_{[0,1]}(x)$; positive at $x = 1/3$	$3\mathbb{1}_{(2/3,1]}(x)$; zero at $x = 1/3$
$Y = \text{given}(U, U > 3/4)$, $\text{density}(Y)(x)$	$4\mathbb{1}_{[0,1]}(x)$; positive at $x = 3/8$	$4\mathbb{1}_{(3/4,1]}(x)$; zero at $x = 3/8$
$Y = \text{given}(U, U > 1/5)$, $\text{density}(Y)(x)$	$\frac{5}{4}\mathbb{1}_{[0,1]}(x)$; positive at $x = 1/10$	$\frac{5}{4}\mathbb{1}_{(1/5,1]}(x)$; zero at $x = 1/10$

These are all instances of the same defect: the normalizing factor changes with t, but the returned support remains the unconditional support $[0, 1]$ instead of the conditional support $(t, 1]$.

1.7 Related Issues Assessment

The best-effort deduplication check found documentation and background material for `density`, `given`, and conditional probability distributions, but did not identify a public SymPy issue, pull request, release note, mailing list thread, or Stack Overflow post for this exact reproducer or the internal `ConditionalContinuousDomain` support-retention failure. The verdict is therefore a new failure mode with no public precedent. The bug remains individually actionable because it has a concise reproducer, a concrete invalid density, a localized source diagnosis, and a focused regression test.

1.8 Proposed SymPy Regression Test

```
import pytest

from sympy import S, integrate, oo, symbols
from sympy.stats import P, Uniform, density, given


@pytest.mark.parametrize("t", [S.Half, S(1)/3, S(1)/4, S(2)/3, S(3)/4, S(1)/5])
def test_conditional_uniform_density_support_is_restricted(t):
    x = symbols("x")
    U = Uniform("U", 0, 1)
    Y = given(U, U > t)
    d = density(Y)(x)

    assert integrate(d, (x, -oo, oo)) == 1
    assert integrate(d, (x, 0, t)) == 0
    assert d.subs(x, t/2) == 0
```

```
    assert P(Y < t) == 0
```